НАО «КАРАГАНДИНСКИЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ ИМЕНИ АБЫЛКАСА САГИНОВА»

Кафедра Кибербезопасности и искусственного интеллекта

Лабораторная работа № 5

по дисциплине «Безопасность Баз данных»

Тема:«Аудит действий пользователей в Microsoft SQL Server с помощью SQL Server Audit»

Специальность: 6В06301 – «Системы информационной безопасности»

Принял: ст.преп.
Жакина М.М.
Выполнил:
СИБ 23-7 Гарифуллин И.Р.

Караганда 2026

Подготовка:

```
DROP DATABASE IF EXISTS audit_demo_db;
CREATE DATABASE audit_demo_db;

CREATE SCHEMA company;
CREATE TABLE company.departments (
   id SERIAL PRIMARY KEY,
   name TEXT NOT NULL
);
CREATE TABLE company.employees (
   id SERIAL PRIMARY KEY,
   full_name TEXT NOT NULL,
   position TEXT NOT NULL,
   salary NUMERIC(12,2) NOT NULL,
   phone TEXT,
   email TEXT,
   department_id INT REFERENCES company.departments(id),
   is_active BOOLEAN DEFAULT true
);
INSERT INTO company.departments(name)
VALUES
('IT'),
('Accounting'),
('Security');
INSERT INTO company.employees(full_name, position, salary, phone, email,
department_id)
VALUES
('Ivan Petrov', 'Developer', 350000, '+77001112233', 'ivan@company.kz', 1),
('Anna Smirnova', 'Accountant', 300000, '+77004445566', 'anna@company.kz', 2),
('Sergey Volkov', 'Security Officer', 250000, '+77007778899', 'sergey@company.kz',
3);

CREATE ROLE user1 LOGIN PASSWORD 'user123';
CREATE ROLE audit1 LOGIN PASSWORD 'audit123';
CREATE ROLE admin1 LOGIN PASSWORD 'admin123';
CREATE ROLE app_role1;
CREATE ROLE admin_role1;
CREATE ROLE audit_role1;
GRANT app_role1 TO user1;
GRANT admin_role1 TO admin1;
GRANT USAGE ON SCHEMA company TO app_role1;
GRANT SELECT, INSERT, UPDATE, DELETE ON company.employees TO app_role1;
GRANT SELECT ON company.departments TO app_role1;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA company TO app_role1;
GRANT USAGE ON SCHEMA company TO audit1;
GRANT SELECT ON ALL TABLES IN SCHEMA company TO audit1;
GRANT USAGE ON SCHEMA company TO admin_role1;
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA company TO
admin_role1;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA company TO admin_role1;

CREATE EXTENSION IF NOT EXISTS pgaudit;

ALTER SYSTEM SET log_connections = on;
ALTER SYSTEM SET log_disconnections = on;
ALTER SYSTEM SET log_statement = 'none';
ALTER SYSTEM SET log_line_prefix = '%m [%p] user=%u db=%d ';
ALTER SYSTEM SET pgaudit.log_parameter = on;
ALTER SYSTEM SET pgaudit.log_relation = on;
SELECT pg_reload_conf();
```

Задания

1. Настроить аудит только для одной выбранной таблицы из своей базы данных.

```
ALTER SYSTEM SET pgaudit.role = 'audit_role1';
SELECT pg_reload_conf();
GRANT SELECT, INSERT, UPDATE, DELETE
ON company.employees
TO audit_role1;

UPDATE company.employees
SET salary = salary + 10000
WHERE id = 1;
DELETE FROM company.employees
WHERE id = 3;
```

Т.к. работа выполнялась в docker контейнере проверка логов осуществляется с помощью:

```
docker logs postgres-anon-audit | grep AUDIT
```

2. Настроить аудит только для конкретного пользователя или роли.

ALTER ROLE user1 SET pgaudit.log = 'write';
ALTER ROLE user1 SET pgaudit.log_relation = on;

проверка под юзером1

UPDATE company.employees
SET phone = '+77009998877'
WHERE id = 1;

```
2026-04-25 07:11:33.720 UTC [107] user=user1 db=audit_demo_db LOG:  connection authorized: user=user1 database=audit_demo_db
2026-04-25 07:11:35.460 UTC [107] user=user1 db=audit_demo_db LOG:  AUDIT: OBJECT,1,1,WRITE,UPDATE,TABLE,company.employees,"UPDATE company.employees
        SET phone = '+77009998877'
        WHERE id = 1",<none>
2026-04-25 07:11:35.460 UTC [107] user=user1 db=audit_demo_db LOG:  AUDIT: SESSION,1,1,WRITE,UPDATE,TABLE,company.employees,"UPDATE company.employees
        SET phone = '+77009998877'
        WHERE id = 1",<none>
```

3. Настроить аудит только операций DELETE и UPDATE.

В pgAudit нельзя включить строго только `UPDATE` и `DELETE`. Настройка `write` включает: INSERT, UPDATE, DELETE, TRUNCATE

Строго `UPDATE` и `DELETE` делаем через триггер.

```
DROP TABLE IF EXISTS company.audit_log CASCADE;
CREATE TABLE company.audit_log (
```

```
    id BIGSERIAL PRIMARY KEY,
    event_time TIMESTAMP DEFAULT now(),
    username TEXT DEFAULT current_user,
    table_name TEXT,
    operation TEXT,
    old_data JSONB,
    new_data JSONB
);
```

Функция:

```
CREATE OR REPLACE FUNCTION company.audit_update_delete()
RETURNS TRIGGER AS $$
BEGIN
    IF TG_OP = 'UPDATE' THEN
        INSERT INTO company.audit_log(table_name, operation, old_data, new_data)
        VALUES (TG_TABLE_NAME, TG_OP, to_jsonb(OLD), to_jsonb(NEW));
        RETURN NEW;
    END IF;
    IF TG_OP = 'DELETE' THEN
        INSERT INTO company.audit_log(table_name, operation, old_data, new_data)
        VALUES (TG_TABLE_NAME, TG_OP, to_jsonb(OLD), NULL);
        RETURN OLD;
    END IF;
END;
$$ LANGUAGE plpgsql;
```

Триггер:

```
DROP TRIGGER IF EXISTS trg_employees_audit_update_delete
ON company.employees;
CREATE TRIGGER trg_employees_audit_update_delete
AFTER UPDATE OR DELETE ON company.employees
FOR EACH ROW
EXECUTE FUNCTION company.audit_update_delete();
```

Проверка:

```
UPDATE company.employees
SET salary = 390000
WHERE id = 1;
DELETE FROM company.employees
WHERE id = 2;
```

Результат:

```
SELECT *
FROM company.audit_log
ORDER BY event_time DESC;
```

123 id	event_time	A-Z username	A-Z table_name	A-Z operation	old_data	new_data
2	2026-04-25 12:15:29.437	app_user	employees	DELETE	{"id": 2, "email": "anna@company.kz", "phone": "+770	[NULL]
1	2026-04-25 12:15:24.454	app_user	employees	UPDATE	{"id": 1, "email": "ivan@company.kz", "phone": "+7700	{"id": 1, "email": "ivar

4. Найти все события за последние 24 часа.

Через pgAudit — только в логах PostgreSQL:

```
docker logs postgres-anon-audit | grep AUDIT
```

5. Найти все действия пользователя, связанные с изменением важного поля или чувствительных данных.

Важные поля: salary, phone, email
Тестовые изменения:

```
UPDATE company.employees
SET salary = 500000
WHERE id = 1;
UPDATE company.employees
SET phone = '+77770001122'
WHERE id = 1;
UPDATE company.employees
SET email = 'changed_ivan@company.kz'
WHERE id = 1;

SELECT id, event_time, username,
      old_data ->> 'salary' AS old_salary,
      new_data ->> 'salary' AS new_salary,
      old_data ->> 'phone' AS old_phone,
      new_data ->> 'phone' AS new_phone,
      old_data ->> 'email' AS old_email,
      new_data ->> 'email' AS new_email
FROM company.audit_log
WHERE operation = 'UPDATE'
 AND (
     old_data ->> 'salary' IS DISTINCT FROM new_data ->> 'salary'
     OR old_data ->> 'phone' IS DISTINCT FROM new_data ->> 'phone'
     OR old_data ->> 'email' IS DISTINCT FROM new_data ->> 'email'
 )
ORDER BY event_time DESC;
```

123 id	event_time	A-Z username	A-Z old_salary	A-Z new_salary	A-Z old_phone	A-Z new_phone	A-Z old_email	A-Z new_email
6	2026-04-25 12:22:18.730	app_user	500000.00	500000.00	+77770001122	+77770001122	ivan@company.kz	changed_ivan@company.kz
4	2026-04-25 12:20:23.671	app_user	500000.00	500000.00	+77009998877	+77770001122	ivan@company.kz	ivan@company.kz
3	2026-04-25 12:20:18.363	app_user	390000.00	500000.00	+77009998877	+77009998877	ivan@company.kz	ivan@company.kz
1	2026-04-25 12:15:24.454	app_user	360000.00	390000.00	+77009998877	+77009998877	ivan@company.kz	ivan@company.kz

6. Найти все случаи неудачного входа.

```
ALTER SYSTEM SET log_connections = on;
SELECT pg_reload_conf();
```

Выполнить неправильное подключение

```
docker logs postgres-anon-audit | grep "password authentication failed"
```

```
2026-04-25 07:26:11.373 UTC [165] user=[unknown] db=[unknown] LOG:  connection received: host=172.18.0.1 port=45820
2026-04-25 07:26:11.414 UTC [165] user=user1 db=audit_demo_db FATAL:  password authentication failed for user "user1"
2026-04-25 07:26:11.414 UTC [165] user=user1 db=audit_demo_db DETAIL:  Connection matched file "/var/lib/postgresql/data/pg_hba.conf" line 128: "host all all
 all scram-sha-256"
```

7. Настроить отдельный аудит для пользователей с административными правами.

```sql
ALTER ROLE admin_role1 SET pgaudit.log = 'all';
ALTER ROLE admin_role1 SET pgaudit.log_relation = on;

CREATE TABLE company.admin_test (
    id SERIAL PRIMARY KEY,
    note TEXT
);
```

```
2026-04-25 07:29:41.266 UTC [36] user=app_user db=audit_demo_db LOG:  AUDIT: SESSION,80,1,ROLE,GRANT,SCHEMA,,GRANT CREATE ON SCHEMA company TO admin_role1,<n
one>
2026-04-25 07:29:49.912 UTC [180] user=admin1 db=audit_demo_db LOG:  AUDIT: SESSION,2,1,DDL,CREATE SEQUENCE,SEQUENCE,company.admin_test_id_seq,"CREATE TABLE
company.admin_test (
            id SERIAL PRIMARY KEY,
            note TEXT
        )",<none>
2026-04-25 07:29:49.912 UTC [180] user=admin1 db=audit_demo_db LOG:  AUDIT: SESSION,2,1,DDL,CREATE TABLE,TABLE,company.admin_test,"CREATE TABLE company.admin
_test (
            id SERIAL PRIMARY KEY,
            note TEXT
        )",<none>
2026-04-25 07:29:49.912 UTC [180] user=admin1 db=audit_demo_db LOG:  AUDIT: SESSION,2,1,DDL,CREATE INDEX,INDEX,company.admin_test_pkey,"CREATE TABLE company.
admin_test (
            id SERIAL PRIMARY KEY,
            note TEXT
        )",<none>
2026-04-25 07:29:49.912 UTC [180] user=admin1 db=audit_demo_db LOG:  AUDIT: SESSION,2,1,DDL,ALTER SEQUENCE,SEQUENCE,company.admin_test_id_seq,"CREATE TABLE c
ompany.admin_test (
            id SERIAL PRIMARY KEY,
            note TEXT
        )",<none>
2026-04-25 07:30:25.942 UTC [27] user= db= LOG:  checkpoint starting: time
2026-04-25 07:30:30.607 UTC [27] user= db= LOG:  checkpoint complete: wrote 47 buffers (0.3%); 0 WAL file(s) added, 0 removed, 0 recycled; write=4.628 s, syn
c=0.025 s, total=4.665 s; sync files=43, longest=0.004 s, average=0.001 s; distance=155 kB, estimate=156 kB; lsn=0/2578478, redo lsn=0/2578440
```

8. Настроить аудит изменений ролей и прав доступа.

```sql
ALTER SYSTEM SET pgaudit.log = 'role';
SELECT pg_reload_conf();

CREATE ROLE temp_user1 LOGIN PASSWORD 'temp123';
GRANT SELECT ON company.employees TO temp_user1;
REVOKE SELECT ON company.employees FROM temp_user1;
DROP ROLE temp_user1;
```

```
2026-04-25 07:29:49.912 UTC [180] user=admin1 db=audit_demo_db LOG:  AUDIT: SESSION,2,1,DDL,CREATE INDEX,INDEX,company.admin_test_pkey,"CREATE TABLE company.
admin_test (
            id SERIAL PRIMARY KEY,
            note TEXT
        )",<none>
2026-04-25 07:29:49.912 UTC [180] user=admin1 db=audit_demo_db LOG:  AUDIT: SESSION,2,1,DDL,ALTER SEQUENCE,SEQUENCE,company.admin_test_id_seq,"CREATE TABLE c
ompany.admin_test (
            id SERIAL PRIMARY KEY,
            note TEXT
        )",<none>
2026-04-25 07:30:25.942 UTC [27] user= db= LOG:  checkpoint starting: time
2026-04-25 07:30:30.607 UTC [27] user= db= LOG:  checkpoint complete: wrote 47 buffers (0.3%); 0 WAL file(s) added, 0 removed, 0 recycled; write=4.628 s, syn
c=0.025 s, total=4.665 s; sync files=43, longest=0.004 s, average=0.001 s; distance=155 kB, estimate=156 kB; lsn=0/2578478, redo lsn=0/2578440
2026-04-25 07:32:32.201 UTC [36] user=app_user db=audit_demo_db LOG:  AUDIT: SESSION,81,1,DDL,ALTER SYSTEM,,,ALTER SYSTEM SET pgaudit.log = 'role',<none>
2026-04-25 07:32:32.203 UTC [36] user=app_user db=audit_demo_db LOG:  AUDIT: SESSION,82,1,READ,SELECT,,,SELECT pg_reload_conf(),<none>
2026-04-25 07:32:32.203 UTC [1] user= db= LOG:  received SIGHUP, reloading configuration files
2026-04-25 07:32:32.204 UTC [1] user= db= LOG:  parameter "logging_collector" cannot be changed without restarting the server
2026-04-25 07:32:32.204 UTC [1] user= db= LOG:  configuration file "/var/lib/postgresql/data/postgresql.auto.conf" contains errors; unaffected changes were a
pplied
2026-04-25 07:32:45.260 UTC [36] user=app_user db=audit_demo_db LOG:  AUDIT: SESSION,83,1,ROLE,CREATE ROLE,,,CREATE ROLE temp_user1 LOGIN PASSWORD <REDACTED>
,<none>
2026-04-25 07:32:45.273 UTC [36] user=app_user db=audit_demo_db LOG:  AUDIT: SESSION,84,1,ROLE,GRANT,TABLE,,GRANT SELECT ON company.employees TO temp_user1,<
none>
2026-04-25 07:32:45.275 UTC [36] user=app_user db=audit_demo_db LOG:  AUDIT: SESSION,85,1,ROLE,REVOKE,TABLE,,REVOKE SELECT ON company.employees FROM temp_use
r1,<none>
2026-04-25 07:32:45.278 UTC [36] user=app_user db=audit_demo_db LOG:  AUDIT: SESSION,86,1,ROLE,DROP ROLE,,,DROP ROLE temp_user1,<none>
```

9. Сравнить аудит SQL Server и триггеры аудита.

SQL Server Audit — встроенный механизм аудита в Microsoft SQL Server. Он позволяет централизованно фиксировать события на уровне сервера и базы

данных: входы пользователей, обращения к объектам, изменение прав, выполнение команд и административные действия.

Триггеры аудита в PostgreSQL — это ручной способ фиксации изменений данных. Они срабатывают при INSERT, UPDATE или DELETE и позволяют сохранить старые и новые значения строки, пользователя, время операции и тип действия.

SQL Server Audit и pgAudit больше подходят для фиксации SQL-команд и системных событий. Триггеры больше подходят для детального аудита данных, когда нужно видеть, какое именно значение поля было изменено.

10. Определить, какие события наиболее критичны для выбранной базы данных или компании

Для базы audit_demo_db критичны:

Изменение salary, phone, email.
Удаление сотрудников.
Массовые UPDATE и DELETE.
Создание новых пользователей.
Выдача прав через GRANT.
Отзыв прав через REVOKE.
Неудачные попытки входа.
Действия admin1.
Изменение структуры таблиц.

Контрольные вопросы:

1. SQL Server Audit — это встроенный механизм аудита в Microsoft SQL Server, который позволяет фиксировать действия пользователей и системные события на уровне сервера или базы данных. Он используется для контроля безопасности, отслеживания действий пользователей и расследования инцидентов.
2. Audit — это объект верхнего уровня, который определяет, куда будут записываться события (например, в файл или журнал Windows). Audit Specification — это набор правил, который определяет, какие именно события нужно фиксировать. То есть Audit — это “куда писать”, а Audit Specification — “что писать”.
3. Можно фиксировать входы пользователей, неудачные попытки входа, выполнение SQL-команд (SELECT, INSERT, UPDATE, DELETE), изменения структуры базы (DDL), изменения ролей и прав доступа

(GRANT, REVOKE), действия администраторов и доступ к конкретным объектам.

4. Результаты аудита могут храниться в файлах на диске, в журнале безопасности Windows (Security Log) или в журнале приложений Windows (Application Log).
5. SUCCESSFUL_LOGIN_GROUP фиксирует все успешные подключения пользователей к серверу SQL Server, то есть когда логин выполнен без ошибок.
6. FAILED_LOGIN_GROUP фиксирует все неудачные попытки входа, например при неправильном пароле или отсутствии доступа.
7. sys.fn_get_audit_file — это системная функция, которая используется для чтения данных аудита из файлов. Она позволяет выполнять SQL-запросы к логам аудита, как к обычной таблице.
8. Наиболее полезные поля: имя пользователя (server_principal_name), время события, тип действия (action_id), имя базы данных, имя объекта (таблица), текст выполненной команды (statement), IP-адрес клиента и результат операции (успешно или нет).
9. Действия конкретного пользователя можно найти, отфильтровав данные аудита по имени пользователя, например по полю server_principal_name, используя запрос к таблице аудита или функции sys.fn_get_audit_file.
10. SQL Server Audit лучше триггеров тем, что он встроенный, централизованный и охватывает больше событий, включая входы пользователей и системные действия. Он не требует написания кода для каждой таблицы и не влияет существенно на производительность. Триггеры же подходят только для отслеживания изменений данных в таблицах и требуют ручной реализации.